Knowledge and Information Systems
https://doi.org/10.1007/s10115-024-02232-1

REVIEW

Automatic regex synthesis methods for english: a comparative analysis

Sadia Tariq[1] · **Toqir Ahmad Rana**[1]

Received: 12 August 2023 / Revised: 16 August 2024 / Accepted: 3 September 2024

Abstract
Regular expressions (short form regex) find their application in program script synthesis, machine translation, information extraction and web applications, such as input validations. Their expressiveness and flexibility make them decidedly the best tool for many challenging text extraction tasks. Writing regex manually has been labeled as a laborious, time consuming and error prone task even for skilled programmers. An abundance of regex generation from text queries at online platforms mainly Stackoverflow[1] and Quora[2] signifies the automatic regex synthesis problem. Despite their popularity, a criminal lack of comprehensive literature study on the problem has also been observed. We intend to perform a detailed review of a variety of methods available for regex synthesis, repair, and learn beneficial lessons for appropriate datasets with one earnest goal: to synthesize resource efficient and correct regexes for given textual description.

Keywords Regular expressions · Regex · Regex synthesis · Neural networks · Probabilistic regular grammar

1 Introduction

Regular expression is a technology widely used in software development for extracting textual data, validating the structure of textual documents, or formatting data. Text extraction-based methods are bent upon finding regular expressions that generalize the extraction behavior represented by string examples. Many top-notch researches in text processing focus on smart and automatic generation of regular expressions for potentially targeting a large number of words in one go within a corpus. In order to perform dataset preprocessing tasks, a method to identify all possible variations of a word within the text is required. **Regular expressions**

[1] https://stackoverflow.com/search? q=regular+expression+generation.
[2] https://www.quora.com/search?q=regular% 20expression%20generation.

✉ Sadia Tariq
sadia.tariq.shah@gmail.com

Toqir Ahmad Rana
toqirr@gmail.com

[1] Department of CS and IT, The University of Lahore, Lahore, Pakistan

Published online: 03 October 2024

Table 1 Regular expressions, their corresponding string examples and textual descriptions

Regular expression	Positive examples	Negative examples	NL-description
*.*s*	fj38fj498js	SIJF$$FES	Accepts a string of characters that ends with an 's'
	r5iffffkkkks	48f94wfwh	
	59yhkgyg7s	GRSRSRSFJh	
	FJEFJISFJs	sw4wfJEHSFK	
\[.]*	[hello]	hello]	Accepts a string of characters that begin and end with square brackets
	[dog]	[hello	
	[cat]	[]hello	
	[hat]	hello[]	

Table 2 Common Meta characters in regex

Meta character	Name	Matches
.	Dot	Matches any one character
[..]	Character class	Matches any one character listed
[^..]	Negated character class	Matches any one character not listed
?	Question	One allowed, but it is optional
*	Star	Any number allowed, but all are optional
+	Plus	At least one required, additional are optional
|	Alternation	Matches either expression it separates
^	Caret	Matches the position at start of line
$	Dollar	Matches the position at the end of line
{X,Y}	Specified range	X required, max allowed

(short form regex) are widely employed to find matching patterns and further replacement within strings. Two simple examples of regular expressions, their corresponding string examples and textual descriptions can be seen in Table 1.

Common symbols, their meanings with examples are revealed in Table 2.

1.1 Regex synthesizer/generator/engine

Regular expressions have proven themselves an extremely powerful and versatile formalism that has made its way into everything from spreadsheets to databases. However, despite

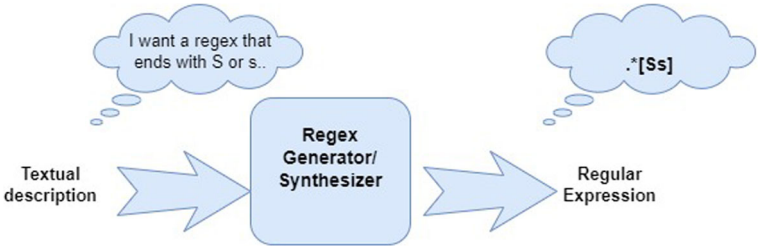

Fig. 1 Regex synthesis block diagram

their usefulness and wide availability, they are still considered a dark art that even many programmers do not fully understand. Thus, the ability to automatically generate regular expressions from natural language (NL) would be useful in many contexts [25]. A regex synthesizer can be imagined as a black box that accepts user written text snippets and/or few example strings as input to produce an expected regular expression as output and is shown in Fig. 1.

The two fundamental goals of regex synthesis problem will always be: (1) production of time efficient regex and (2) production of correct/valid regex for a given text specification. Our interest behind working on efficient and valid automatic regex generation is its rapid and less resource consuming capability, suitable for various text extraction tasks.

1.2 Study design and selection process

Exhaustive survey has been conducted by skimming various scientific journals of relevant domains. Studies for automatic regex synthesis have been split into three major divisions; learning-based, rule-based and grammar-based methods. We explored Google Scholar by using keywords given above. No time limit was set for the inclusion of an article in the study. We tried to involve as many articles as possible because of a lack of existing survey on chosen topic. Starting with modes and methods-based division; we collected articles from various sources including IEEE access and ResearchGate and from repositories such as ArXiv for pre-prints and e-prints. Total 26 core papers were selected and divided into 9 different categories based on the similarity in their way of producing regex. Another 7 papers and 2 dissertations were included in the study in pursuit of knowing the recent trends in the field. These were picked from Google Scholar from year 2023 onwards using keywords: LLM and GPT for autoregex synthesis. Figure 2 explains the complete procedure of methodology of survey.

1.3 Methods of regex synthesis

Three kinds of available regex synthesis method have been discussed in Zhong et al. [64] : (1) **rule based, (2) probabilistic regular grammar (PRG) based** to get regex from parse tree created from NL description and **(3) neural model based** using seq2seq learning model with attention mechanism considering regex synthesis a direct machine translation task**.** More **neural network (NN)-based models** include: neural machine translation [1], fast and robust neural network joint models for statistical machine translation in [14], seq2seq learning in [55] have been demonstrated with promising results.

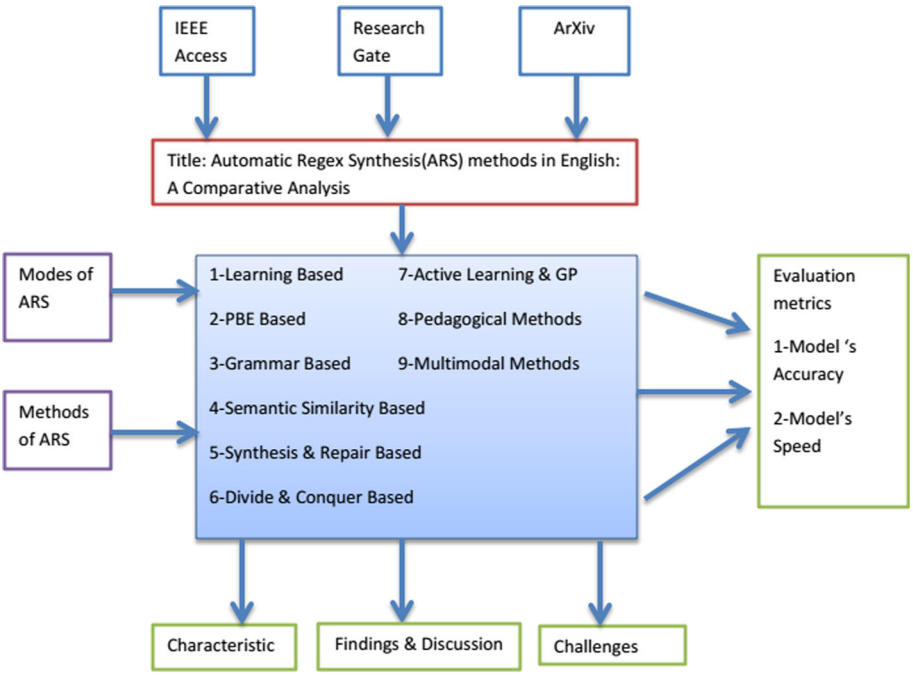

Fig. 2 Methodology of survey

1.4 Modes of regex synthesis

There are three modes of regex synthesis named: (1) purely NLP based, (2) purely string examples based and (3) multimodal method. NLP descriptions are confined by imprecision and ambiguity [28] Example string-based synthesis approaches rely on quality of user provided examples. Synthesized regex may suffer from under fitting and over fitting issues due to insufficient examples and not characteristic enough examples, respectively. Moreover, they generate restricted regexes. Regex synthesis approaches are sensitive to incorrect, sloppy, complicated regexes causing unexpected sequences in practice. Even a single character mistake in regex can make it accept or reject an entirely different set of strings. The methods and modes have been summarized in Fig. 3.

The contributions of our research are threefold and are narrated below:

1. A detailed literature review of regex synthesis and repair frameworks/ methods, their contributions and limitations
2. A discussion on structure and design of suitable dataset for faster and more accurate regex generation
3. A list of suggestions for improvement of overall regex synthesis process

In the upcoming sections, we shall perform detailed survey of Regex Synthesis and Repair Methods, the characteristic properties of such models and their associated challenges. We shall see how models tend to generate more correct and resource efficient regex for given NL query. Afterward we shall discuss the structure and design of more suitable datasets for Regex Synthesis problem. Lastly, we summarize our conclusive remarks along with emphasis on some exciting recommendations for future work.

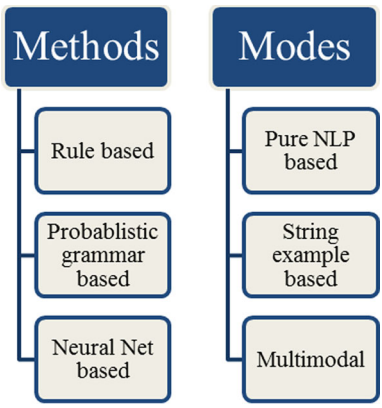

Fig. 3 Methods and modes of regex synthesis

2 Literature survey of regex synthesis methods

According to Ouyang et al. [37], there are four main categories of related work on regex synthesis problem: programming by example, grammar induction and learning regexes from language, nonparametric model of sequence data.

1. **Programming By Example (PBE)-based methods** perform mapping strings onto strings as common approach using example strings and input output pairs [52]. Locascio et al. [32] preferred to learn classifiers that map strings on to Booleans causing reduction in workspace with lesser data.
2. **Grammar induction** is just like inferring a grammar, MAP search [9] is common method in search of one true grammar that generates all data, also merging approaches is a method where only positive examples are generated reducing the search space.
3. **Learning regex techniques** mimic humans by using NL descriptions to generate regexes for example number parser[25] and Locascio et al. [32].
4. **Non parametric models** include Infinite hidden markov model (IHMM) [5]and Problem driven iterative adaptation (PDIA) [42] models which assign some positive probability to all string however better models want classifier to learn by accepting some strings (positive ones) and rejecting others (negative ones).

During our survey, we came across diverse range of methods for regex synthesis that can be further classified into nine different frameworks. This classification is based on the mode and methods they utilize for regex production. A diagrammatic representation of regex synthesis methods has been given in Fig. 4.

2.1 Older regex synthesis methods

These can be counted as:

- **AlphaRegex** which use enumeration-based algorithms
- **Regexgenerator + +** which does not guarantee correct solution
- **Genetic Programming(GP)** regex golf [2] which is genetic programming-based method to generate shortest regex given a set of sentences to perform matches and another set not to match.

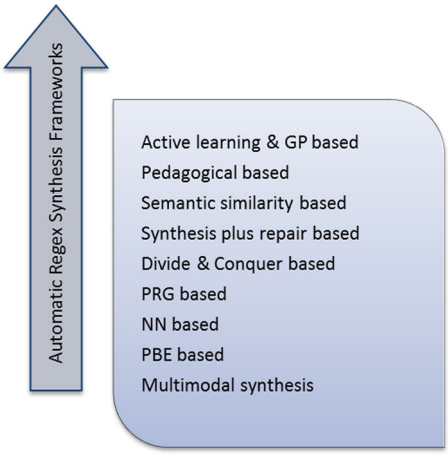

Fig. 4 Automatic regex synthesis frameworks

Taking a newer dig at the problem, in [13] the researchers proposed a heuristic search algorithm based on local search, combined with a regex shrinking mechanism, to find valid results for Regex Golf problems. Lots of existing works focus on XML schemas inference (Bex et al. [7, 15, 16, Fernau27]), via resorting to infer regexes from examples. These approaches usually aim to tackle restricted forms of *deterministic* regexes [15] from positive examples only. Also in [4] they set bases on entity extraction from unstructured data based on active learning and GP. Its ultimate goal is to avoid costly and error prone manual annotation effort.

2.2 Active learning and genetic programming-based synthesis

In active learning user annotates only one desired extraction and then merely answers extraction queries generated by the system. Every part of dataset is considered a potential candidate query and dataset is a single, possibly long, input text without any native segmentation in smaller units, several extractions can exist per line or one extraction can consist of multiple lines. The size of snippet given to user does matter because if the size is small, user will be asked more often to annotate. On the other hand, if size is larger, the effort reduction objective will be minimized.

2.2.1 GP regex golf

GP regex golf by Bartoli et al. [2] is GP-based method to generate shortest regex given a set of sentences both positive and negative. The term Regex golf is a form of code golf that involves creating the shortest regular expression to match all strings in one list while excluding those in another. This expression must adhere to the syntax of a specific language, like JavaScript or PHP. In this setup, regex golf player based on GP is introduced which evolves candidate regular expressions represented as trees to minimize errors and expression length. The experimental results on a well-known regex golf challenge of 16 problems show that this newly introduced player surpasses the baseline and is competitive with human players. Solution times typically range in the tens of minutes, which is comparable to human performance.

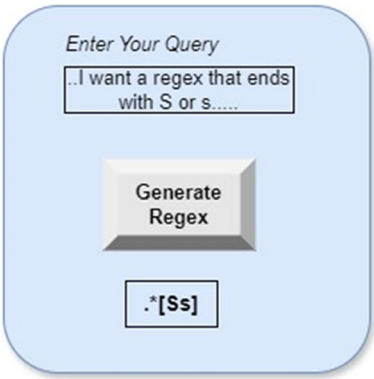

Fig. 5 Query builder/regex generator

2.2.2 Regex-based entity extraction

The framework in [4] sets its bases on entity extraction from unstructured data based on **active learning and Genetic Programming(GP)**. Its ultimate goal is to avoid costly and error prone manual annotation effort. In this framework user initially marks *only one* snippet of the input text as desired extraction. A learner based on Genetic Programming then constructs a solution, sifts the (possibly very long) input text, selects the most appropriate snippet to be used for improving the current model and presents it to the user as an *extraction query*. User answers with yes or no based on whether they want to extract it or not.

In essence, user may modify the received query slightly and then answer the modified query because we offer web-based prototype with GUI for the proposed interaction model. The aim of this design was to help the user in deciding when to put an end to the ideal regex search. This normally happens when the user is satisfied with the current solution. One such hypothetical GUI is shown in Fig. 5.

In future, following studies might be sumptuous for such methods incorporation of user annotation time as a function of the number, length and complexity of individual queries

- Formulation of a suitable metric for taking into account elapsed time between consecutive queries
- Online estimate of difficulty of obtaining a single regex

2.3 Pedagogical regex synthesis

Pedagogical regex synthesis method use positive and negative examples that accept all positive strings and reject all negative ones.

2.3.1 AlphaRegex

AlphaRegex [26] is based on exhaustive search by prioritizing simpler regular expressions; enumeration of all possible regular expressions and checking whether each candidate is consistent with all examples or not. Over- and under-approximations have been used to effectively prune out a large search space and tell whether current solution is the final one or not.

It has been evaluated on 25 non-trivial benchmark specs given by student assignments and exists as a publically available tool. On average, it produces regex within 6.7 seconds synthesized from a few examples quickly so that students can interactively use the tool. Therefore speed of regex production is its notable plus point.

2.3.2 Regexgenerator + +

The next framework, named as **Regexgenerator++,** is loosely based on Alpharegex. The heart of this research work [3] lies in regex synthesis based solely on examples. Positive and negative sample strings represent strings described by the language and strings not described by the language respectively. Practically, entity extraction from small sized and complex text is challenging task. Using Genetic Programming-based heuristic search, their target is to reduce the search space of candidates considerably.

It has been tested on challenging datasets and with 70 human operators. It has been applied to 20 challenging extraction tasks of realistic sizes and complexity, with a very small portion of the dataset available for learning. As a result, it was observed that precision and recall is modest and practically usable.

In future research around this kind of models can focus on faster learning, interactive learning procedures capable of starting with a very small number of snippets and even at achieving better accuracy.

2.3.3 Efficient enumeration strategy

Kim et al. [21] explore problem of synthesizing regular expressions from a set of positive and negative strings. As an improvement in best-first enumeration of regular expressions they define a new normal form of regular expressions called the concise normal form which allows significant reduction in the search space by trimming those not in the normal form.

2.4 Semantic similarity-based synthesis

Semantics-based approaches believe in the equivalence of two model outputs based on the similarity of their corresponding Deterministic Finite Automata (DFA) rather than using string examples.

2.4.1 SemRegex

The model proposed in Zhong et al. [65] is named as **Semregex** which simply convert the output regex into a minimal DFA and compare it with the DFA of the ground truth regex. String example based regex equivalence determination mechanisms use test cases for matching. They suffer from serious limitations in regex domain, no matter how useful they are in other programs script domains.

MLE has a problem that it do not understand that syntactically different regexes can be semantically similar. Though they preferred to pre-train model using MLE but Monte Carlo estimate was chosen to compute gradient of expected reward and reinforcement technique of policy gradient is applied. For each pair of NL specification/regex in train set, they sample M regexes to estimate the gradient.

It was evaluated on three benchmark datasets (NL-RX Turk, NL-RX Synth and KB13). It outperformed state-of-the-art approaches available for the regex synthesis problem. If the

semantic correctness of the output regex is acceptable, then it does not matter if they do not exactly match the ground truth regex (Most likely regex).

2.4.2 Softregex

SoftRegex [39] is based on regex equivalence problem focused in its baseline predecessor named **Semregex**. Finding equivalence of two regexes is P-space complex and time intensive acting as major bottleneck in reducing learning time. A model named **Eq_reg** that computes similarity using deep neural networks was presented. This model softens the equivalence when used as reward function substantially reducing train time by a factor of 3.6 and producing better results on three benchmark datasets.

A basic issue in learning-based methods of regex synthesis is that they produce good regex for train set and can generate regex similar in shape to train set only. **Softregex** can produce regex that may not resemble the ground truth regex but is correct. Most recent learning-based seq2seq model equipped with attention layer uses an encoder to generate latent vector, which is fed as input to decoder, which generates output.

MLE from popular NN model **DeepRegex** and policy gradient by **SemRegex** were used. The regex equivalence-oracle test used in **SemRegex** made it time intensive. Deep learning-based speedup equivalence test returns equivalence probability of two regexes as output. It input regex to two sequences of embedding vectors fed into two LSTMs. They convert them into latent vectors. Using bidirectional RNN, one for forward hidden sequence and another for backward hidden sequence concatenation of latent vectors take place to make single vector containing information of both regexes. Lastly, they predicted equivalence of this concatenated regex vector instead of two separate regexes.

2.4.3 Approximate matching method

Rebele et al. [49] argues that handcrafted regexes may fail to match all the intended words. They propose to generalize a given regex so that it matches also a set of missing words. By finding an approximate match between the missing words and the regex, and by adding disjunctions for the unmatched parts appropriately, they improved the precision and recall of the regex. In addition, they generated much shorter regexes than baselines and competitors on various datasets.

2.5 Synthesis & repair frameworks

These methods synthesize and repair regex in parallel, starting from poor, minimal regex and ending at exact, full fledge regex.

2.5.1 ReScue

In [12] an empirical study of three major aspects of Regex Denial of Service (RE-DOS), most importantly, the incidence of super-linear regexes in practice has discovered that conventional wisdom for super-linear regex anti-patterns has few false negatives but many false positives; these anti-patterns appear to be necessary, but not sufficient, signals of super-linear behavior. Moreover, developers knowingly tend to favor revising them over truncating input or developing a custom parser. Another work presented in [51] proposes **ReScue**, a three-phase gray-box analytical technique, to automatically generate ReDoS strings to highlight

vulnerabilities of given regexes. **ReScue** systematically seeds (by a genetic search), incubates (by another genetic search), and finally pumps (by a regex-dedicated algorithm) for generating strings with maximized search time. It found 49% more attack strings compared with the best existing technique.

2.5.2 Flashregex

FlashRegex [29] model's central idea is the use of deterministic regex which while matching a string from left to right in a regex, considers a symbol in the string to be matched only to one position without any look ahead. This results in efficient matching mechanism. It uses heuristic strategy called local constraint strengthening for further speedup. Three anti-patterns were identified to be the root causes of RE-DOS proneness: (1) regexes with nested quantifiers, (2) ones with overlapping disjunction (QOD) and (3) ones with overlapping adjacency (QOA). Determinism was used to fight ambiguities caused by these nested quantifiers. It has been tested on five state-of-the-art tools. Older tools generated 394 RE-DOS vulnerable regexes within few seconds, whereas **FlashRegex** generated no Super-linear complexity regex within 5 seconds. This performance is even better than human experts, plus its user friendly, general and automatic.

2.5.3 SplitRegex

Kim et al. [23] proposed Splitregex**,** a search and repair strategy and efficient generation of regex, divide and conquer framework. The fundamental goal is to avoid inherent p-space complexity of regex synthesis. Deep neural networks tend to outperform classical approaches of learning regex in terms of accuracy but they are time intensive. To learn faster, trained neural network-based example splitting was utilized. By splitting example string into many parts, group similar substrings were grouped together out of the positive strings. Instead of learning from one long string, the model learned from multiple short substrings and then concatenated the multiple regexes generated by them. For negative cases, pre-generated regexes were used to perform matching against negative strings.

This model embeds set of positive string into a fixed size vector and then produces most probable split for each example in the set. It is a two stage attention-based RNN. It has been tested using four benchmark datasets. It is already known that NL descriptions are ambiguous but provide complete user intent, whereas example strings do not provide complete user intent but they are specific. Somehow being able to use them both will yield the ideal solution.

Most approaches fail when example size is large because they become time intensive. However, they generate correct regex in the end. As alphabet size increases, performance improves because it is easier to group similar substrings having common symbols, as there are more symbols in string. For n-1 out of n sub-regexes, synthesis in parallel with prefix conditional synthesis is tested as compared to independent sequential synthesis where the later outperformed former. Thus, **SplitRegex** guaranteed to yields more accurate regexes more quickly.

2.5.4 Regis and Sygus

McClurg et al. [35] automated the removal of inefficiencies in regular expressions by introducing a new approach called rewrite-guided synthesis (ReGiS), which combines Syntax-guided synthesis (SyGuS) with equality saturation-based rewriting. This unique interplay overcomes

cost and saturation challenges, resulting in an efficient and scalable framework for expression optimization.

2.6 Divide & conquer-based synthesis

Splitting a problem into smaller subproblems is a classical approach, which aims to find partial solutions and then combines them to make one global solution. This is called divide and conquer-based synthesis.

2.6.1 PTM + CBS

Rahmani et al. [45] jointly employ Pre-trained model (PTM) to generate candidate programs from NL clip and component-based system (CBS) to rank them from best match to worst match with the examples. This approach is domain agnostic as opposed to various most recent works which are domain specific. It has been evaluated on two domains: (1) regexes and (2) CSS selectors.

PTM can be used to return multiple possible regexes for one NL specification, sampling them from probability distribution over given specifications. All candidate programs are incomplete and composed of relevant expressions and relevant operators. With the iterative composition of these pieces, more correct regex/program script can be produced. The following Table 3 demonstrates through example.

PTM are powerful but limited precision is their drawback. It was suggested to use multimodal framework for clearer user intent recognition. CBS on the other hand is enumerative search in the space of possible candidate program scripts/regexes. One drawback is the space explosion due to exponential growth in set of candidates. The proposed model use PTM along with CBS leveraging the output of PTM to guide all three key phases of CBS: (1) initializing components, (2) iterative synthesis and (3) ranking of final candidates. CBS improves precision issue of PTM and PTM reduces search space blowup of CBS thus the two methods complement each other.

They measure how effectively and quickly two systems (**NLX-REG** is proposed system and **Regel** is a commercial regex synthesizer) converge to ground truth regex with or without using additional rounds of examples. On average, NLX-REG used 1.5 rounds and regel

Table 3 Many possible candidates for one true regex

Text description	Ground truth regex	PTM's candidates
At least one digit followed by character: at most once followed by a digit at least zero times	[0–9] + :?[0–9] *	([0–9] * ..:([0–9] *)?) + ([0–9] * ([:][0–9] *)) * (0[0–9] +) ([0–9] + :)?[0-9]? [0–9]{3} ([0–9]?: [0–9]?) * ([0–9] * ..: * [0–9] * 0 *) * ([0–9]{1,}(?:.[0–9]{0,})) * ([0–9]{3}) +

required 2.3 rounds of additional examples before guessing the ground truth completely. NL-REG guessed correctly on first trial for 22 cases, whereas Regel did it for two cases only. As future work, one can fine tune PTMs and use sketches, which are terms with holes to guide user input during multimodal interaction.

2.6.2 Example splitting strategy

Kim et al. [22] present an effective regex synthesis framework that constructs sub-regexes from segmented positive substrings and combines them to form the ultimate regex. Leveraging pre-generated sub-regexes during negative sample analysis ensures consistency with all negative strings, making a divided-and-conquer approach that significantly enhances regex synthesis accuracy across four benchmark datasets compared to previous methods.

2.7 PRG-based synthesis

These methods infer regular expression by incrementally growing grammar using positive and negative string examples and then convert them into regex.

2.7.1 Bayesian approach

By using Bayesian approach [37] they assigned high prior to shorter regexes, higher likelihood to good regexes (that explain the example string well) and perform approximate inference. In this way, the user-generated strings' uninformative nature issue is resolved. As a solution to inefficient use of computational resources, stochastic process recognition model is used that causes inference to be partial toward high posterior by incrementally parsing string examples.

As an experiment, human users were given a list of candidate regexes to recover the target regex; top K regex list is given, as human's attention is limited up to certain regexes. As K increased human ability to recover the target regex decreases. K-best score means top K score datasets are those good datasets for which almost all humans recover target regex and bad ones are those for which no human was able to recover the target from.

It was observed that in comparison to human learners, where they set threshold that if 50% humans fail to recover the target regex from given example strings then it will be considered poor human performance, the algorithm succeeds 80 % of the times. The experimental cases have been categorized as those where humans outdid the algorithm and vice versa. Reason for human learners overpowering the algorithm turned out to be pedagogical order of example strings that helped humans foresee the target regex clearly. On the other hand, the reason for algorithm overpowering the human learners is limitedness of short-term memory and short period of focused attention of human brain.

2.7.2 StructuredRegex

Chen et al. [8] suggested **StructuredRegex**, a dataset for regex synthesis which is unique in three aspects: (1) Regexes are generated using probabilistic grammar with pre trained macros gathered from real-world StackOverflow posts, (2) Diverse NL descriptions rather than paraphrasing from synthetic language and (3) Each regex is provided with multiple strings both positive and negative based on ground truth regex. It is a large scale, realistic dataset with triplet information: (1) NL snippets, (2) Example strings and (3) Regexes. Therefore, it is truly multimodal in nature.

2.8 NN-based synthesis

These methods utilize the power of neural networks and deep learning from manually labeled dataset to generate regex for unseen input.

2.8.1 KB13

Kushman's **KB13 model** [25] utilizes domain specific components which compute the semantic equivalence between two regexes, cannot be applied to other formal representations where such equivalence calculations are not possible. The key idea is to translate NL text into regexes considering it a direct machine translation task. In addition, it has to be scalable enough to translate text into more formal representations like program scripts.

2.8.2 DeepRegex

Famous domain agnostic neural model named **Deep-Regex** has been presented in Locascio et al. [32]. **DeepRegex** is long short-term memory (LSTM)-based sequence-to-sequence neural network with two stages generate and paraphrase procedure. In generate phase, small manually crafted grammar was used to translate regex into NL. During paraphrase phase, rigid descriptions were converted to descriptions that are more natural. It has been discovered that model accuracy is higher for smaller dataset than larger KB13, which is much less varied and complex dataset than NL-RX KB13 has repetitions up to 45 % while NL-RX contains up to 97% unique regexes. In addition, KB13 acquires high performance on NN baselines due to its easiness.

It used large corpus of regex and strings pairs for regex synthesis problem. Novel Dataset thus created is named NL-RX with 10,000 NL-Regex pairs. Performance gain of 19.6% has been achieved. It subsequently employed a generate-and-paraphrase procedure given in [60] to create NL-TURK dataset. However, the regexes in this dataset are very simple, and the descriptions are short, formulaic, and not linguistically diverse because of the paraphrasing annotation procedure [20].

2.8.3 Smore

Chen et al. [10] introduce semantic regexes, a broader form of regular expressions that enables both syntactic and semantic analysis of textual data. A learning algorithm implemented in a tool named **Smore**, efficiently synthesizes semantic regexes from limited positive and negative examples through a combination of neural sketch generation and compositional type-directed synthesis, demonstrating effectiveness in various data extraction tasks. The neural guided synthesis algorithm uses large language model (LLM).

2.8.4 REDEx

Redd et al. [50] introduces REDEx, a method that uses regex discovery for automated data extraction with minimal effort. In tasks like body weight extraction from electronic health records (EHRs), REDEx consistently outperforms support vector machines (SVMs). The dataset includes pairs of snippets retrieved with keywords and regexes generated by REDEx.

2.9 PBE-based synthesis

A model that synthesizes rules from examples like input-output pairs is advantageous if the end user does not know how to program rules. Alternatively, if he has clear examples in mind but wishes to outsource the burden of writing the rule. This is known as programming by example.

2.9.1 Regel

The work presented by [8] performed synthesis of regex by jointly employing NL and examples proposing new framework named regel. **Regel** at first parses the English description into a so-called *hierarchical sketch* to direct a PBE engine. Since the hierarchical sketch captures crucial hints, the PBE engine can leverage this information and prunes and prioritize the search space using deductive reasoning of its own.

It has been tested on over 300 regexes of all shapes and sizes. Regel achieves 80% accuracy, whereas the NLP-only and PBE-only baselines achieve 43% and 26% respectively. It has been evaluated with 20 human participants to show that it is twice likely to produce the desired regex as compared to other state-of-the-art tools. As a baseline for comparison, **AlphaRegex** was used.

Among the unexplored dimensions of active learning getting hints from user to guide and choose among multiple possible correct regexes has a bright future**. Regel** is based on top k best regexes from among all produced candidates; however, an overview or approximation of the unused candidates can be valuable. Lastly, use of semantic parsing kind of NLP-based methods can give helpful feedback to user if sketch is too crude.

2.9.2 Augmented examples

Zhang et al. [63] addresses the persistent challenge of ambiguity in user-provided examples within PBE. Introducing an interaction model, they reduce cognitive load by offering two types of augmentations to user-given examples in regex: user oriented semantic augmentation and synthesizer oriented data augmentation. A user study demonstrates that interacting with augmented examples significantly enhances task completion rates, reduces time, and increases users' confidence.

2.9.3 Regex +

Pertseva et al. [40] argues regular expressions are a popular target for programming by example (PBE) systems, which seek to learn regexes from user-provided examples. One possible solution is to input the two examples into a regex synthesizer and get a pattern-matching expression. Their framework is named Regex+ and uses positive string example guided synthesis of regex.

Vaithilingam et al. [58] argues that user-specified examples often introduce ambiguity and to address this challenge framing program synthesis tasks as pragmatic communication achieves promising results in a graphics domain. Extending this to regular expressions and rigorous examination of the usability of pragmatic communication, they found that end-users could more successfully convey a target regex using positive examples.

2.9.4 R3

Chida et al. [11] highlights that recent PBE methods for regex synthesis only cater to membership testing, leaving extraction as an open problem. Their method is implemented in a tool called **R3** (Repairing Regex for extraction) and allow substantial reduction in the search space through novel pruning techniques.

Some similar examples include generation of SQL queries [57], spreadsheet formulas [18] and java expressions [19] and bash formulas [31] from NL descriptions.

2.9.5 Multimodal synthesis

NLP-based methods are limited to the shape of train set and can be called train set dependent, having relatively low accuracy on simple benchmark datasets. NLP descriptions have limitations of imprecision and ambiguity. Example string-based synthesis approaches rely on quality of user provided examples but they can help disambiguate NL sentences [34, 48]. Synthesized regex may suffer from under fit and over fit issues due to insufficient and not characteristic enough examples, respectively. Moreover, they generated restricted regexes like missing or having limited characters and binary alphabets.

The ideal approach is to use them both collaboratively that is multimodal approach since the two modes complement each other very well. NLP basis will alleviate the need of user effort to generate characteristic examples, whereas use of examples can disambiguate the errors in the NL descriptions.

2.9.6 TransRegex

TransRegex [28] is an automatic multimodal regex generation method from both NL descriptions and example strings. We can say that basic approach here is NL complimented by examples to disambiguate the NL. NL improves generalization and example strings incorporate specificity. Adding NL clip to examples drastically narrow down search space.

It is more of a synthesis and repair framework rather than simple synthesis. It has been evaluated on 10 relevant tools and 3 benchmark datasets. The accuracy of **TransRegex** is 17, 35 and 38 percent higher on the 3 datasets than other NLP-based approaches. As compared to most recent multimodal methods, accuracy of Transregex is 10-30 percent higher. However **TransRegex** can generate 100 percent valid regexes on complex datasets, whereas purely NLP-based methods can synthesize 49-90 percent valid ones.

2.9.7 Brief survey of regex repair methods

Regex repair models can be broadly categorized into two major sets:

- The ones which use only positive or negative examples for repair
- The ones which use both kinds together

Various techniques have been proposed to identify ReDoS-vulnerabilities, which can be mainly classified into two paradigms: static analysis [24, 46, 47, 53, 61] and dynamic fuzzing [41, 54].

REbele [49] generalize a regex so that is can accept positive examples. Repairing RE based on Neighborhood search and repair incorrect RE-DOS vulnerable regexes using automaton

directed repair. **Syncorr** on the other hand uses NS and regex directed repair, forces some rewriting rules for sub regex abstraction to preserve integrity of small sub-regexes.

Among the recent regex repair methods **RELiE** [27] modify complex regexes by rejecting newly input negative examples. **RFixer** [38] is popular for its fixing the incorrectness issue of regexes. Given an incorrect regex and positive and negative examples, **RFixer** generates the closest correct expression. It efficiently navigates the large search space by using structural properties and satisfiability modulo theory solvers. It provides minimal repairs for regexes from diverse sources, outperforming existing tools that either fail or produce overly complex fixes.

ReDoS attacks can also be alleviated by regex matching speedup, which is possible in some special cases like by parallel algorithms [30], GPU-based algorithms [62], state-merging algorithms [6], and famous 1968's Thompson's Non-deterministic Finite Automaton algorithm.

3 Findings and discussion

First, let us have a look at the dataset utilization and category wise distribution of the methods discussed above. The utilization of popular datasets is given in Fig. 6.

The category wise distribution of automatic regex synthesis methods is evident in Fig. 7.

From Figs. 6 and 7, the following information is obvious:

- KB13 is the most popular realistic dataset used for regex synthesis
- NL-RX-Synth is the most popular synthetic dataset for the purpose
- Synthesis & repair-based approach has been researchers' favorite for regex synthesis
- Semantic similarity and pedagogical-based methods follow

A summarized version of the surveyed methods is narrated in Table 4.

Following is a concise list of results obtained from the aforementioned methods. They are categorized based on accuracy and speed of regex generation and models that worked on datasets too.

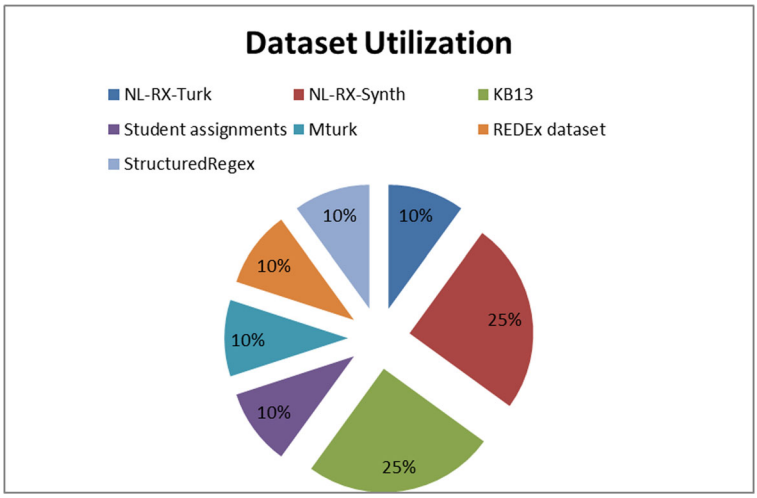

Fig. 6 Dataset utilization [68]

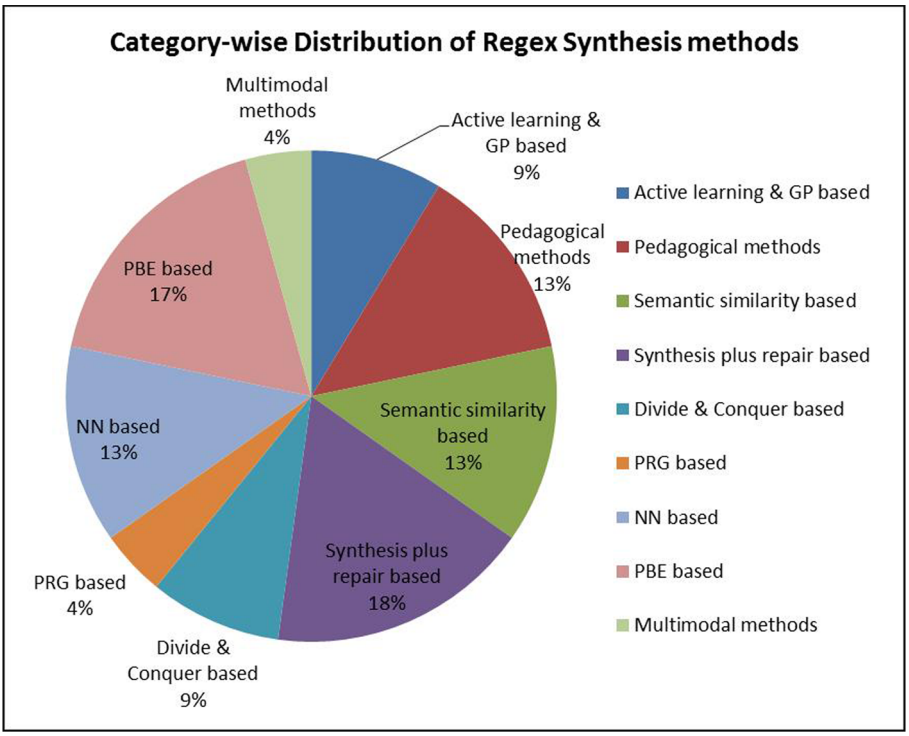

Fig. 7 Category-wise Distribution of regex synthesis methods

- Speedy regex generation
  - GP based **AlphaRegex** [26] has worked around speed and produced regex within an average 6.7 s.
  - **SoftRegex** [39] has reduced training time for learning-based methods by a factor of 8.8(from 283 to 32 s approx.). The least train time was observed to be 4.18 s for KB13 dataset, which dropped from 56.7 s to 4.18 s.
  - For generate and repair-based methods, no regex with Super-linear complexity was generated within 5 s where older tools used to generate 394 vulnerable regex within few seconds in **FlashRegex** [29]. It dropped average synthesis time to 1.9 s from 7.6 s for **AlphaRegex** baseline and to 4 s in the worst case.
  - NLX-REG in **PTM plus CBS** based method [45] used 1.5 rounds, whereas **Regel** (baseline method) required 2.3 rounds of additional examples to converge to the ground truth regex completely. Therefore, it is 0.8 times faster.
  - Though **SoftRegex** appears to have the greatest increase from baseline, the clear winner in terms of speed is **FlashRegex**. A pictorial comparison of these methods is given in Fig. 8.
- Accurate regex generation
  - **PTM plus CBS** is not only considerably faster but also around 23% more accurate too. Across both data sets, it solves 80% of the tasks, as compared to the closest baseline **Regel** that solves 57% tasks.

Table 4 Summary of regex synthesis models

References No	Model	Category/novelty	Dataset/tools for evaluation
[26]	**Alpharegex**	A search-based pedagogical regex synthesis method	25 non-trivial benchmark NL specifications from student assignments
[28]	**Transregex**	A multimodal regex generation & repair	Evaluated on 10 relevant tools and 3 benchmark datasets
[29]	**Flashregex**	An efficient matching mechanism	Tested against 5 state-of-the-art tools
(Zhong, Guo, Yang, Peng, et al., 2018)	**SemRegex**	A semantic similarity-based model	3 benchmark datasets: NL-RX-Turk, NL-RX Synth and KB13
[39]	**SoftRegex**	A successor of **SemRegex** avoiding MLE and its issues	3 benchmark datasets: NL-RX-Turk, NL-RX Synth and KB13-
[8]	**Regel**	PBE-based method	Tested on over 300 regular expressions of all shapes and sizes
[3]	**RegexGenerator + +**	Based on **Alpharegex**	Tested with 70 human operators
[21]	**SplitRegex**	A search and repair strategy based	Random datasets
[32]	**DeepRegex**	A Domain agnostic seq2seq learning model using MLE	Novel NL-RX
[37]	**Probabilistic grammar based**	Bayesian approach using likelihood estimate and stochastic process recognition model	Example datasets generated by Amazon Mech-Turk workers
[45]	**PTM + CBS**	A Domain agnostic, Divide and conquer-based strategy	Tested against Regel
[25]	**KB13**	A Domain specific, semantic equivalence-based method	Novel KB13
[8]	**StructuredRegex**	PRG-based regex synthesis	Novel STReg
[50]	**REDEx**	NN (SVM)-based regex synthesis	EHR Clinical Dataset
[10]	**Smore**	LLM plus neural sketch generation	–

- The accuracy was similar to **SemRegex** (91%) but better than **DeepRegex** (89%) in the best case for **SoftRegex**. Only 2% improvement was offered.
- Semantic similarity based **SemRegex** outperforms NN based **DeepRegex**, the existing state-of-the-art approach, by an accuracy increase of 12.6% on KB13 as best case.
- In multimodal methods, **TransRegex** [28] offered maximum 38% percent higher accuracy other NLP-based approaches. In addition, its accuracy was 10–30 percent higher among the most recent multimodal methods.
- PBE based **Regel** [8] has achieved 80% accuracy, whereas the NLP-only and PBE-only baselines achieved 43% and 26%, respectively, offering 54% improvement.
- **RegexGenerator + +** [3] managed to generate modest and practically usable scores for precision and recall with 60% improvement in F-score.
- As alphabet size increases, performance of **SplitRege**x [21] improved. It also boosted accuracy to 61.6% from 44.8% using **AlphaRegex** as baseline offering 17% improvement.
- **PRG** based model [37] succeeded 80% of the times in generation of correct target regex when only less than 50% humans recovered the said regex.
- As a benchmark NN-based model, performance gain of 19.6% has been achieved by **DeepRegex** Locascio et al. [32] as compared to BoW-NN and Semantic Unify baselines. The accuracy was improved from 30.6%(for BoW-NN on NL-RX-Synth dataset) to 88.7%, which is almost 3 times better.
- The clear winner in terms of correctness is **DeepRegex** offering 58% improvement from its baseline. **SemRegex/SoftRegex** was even better than **Deepregex,** giving 2–12.6 percent accurate results. These results have been visually presented in Fig. 9.

- Datasets
  - Among the dataset-based work, **KB13** [25] has been found to have repetitions up to 45% while **NL-RX** has up to 97% unique regexes.
  - KB13 acquires high performance on baseline neural nets with 29% improvement in accuracy. It outperforms exact string-based unification by over 30%, example-based semantic-unification by over 13% and smart heuristic-unification procedure by 9%.
  - Zhong et al. [64] performed real versus synthetic dataset debate highlighting the very low effectiveness of learning-based regex generation methods for real datasets.
  - The recent datasets offer synthetic instances, whereas the ideal dataset would be a balanced mixture of real and synthetic instances. Models need to be improved to handle realistic datasets as well.

Based on comprehensive literature review of the regex synthesis problem, the redirection of future work to synthesis plus repair frameworks is deemed correct for speedy synthesis. NN and semantic similarity-based methods produce more correct results and therefore need to be elaborated further. Rule-based approaches have not been fondly pursued in contemporary architectures due to the weakness of rules and performance issues.

4 Current trends: LLMs and GPT

Engaging large language models (LLMs) like GPT for automatic regex synthesis opens up several promising applications.

1. They can generate regex patterns from natural language descriptions, making it easier for users to create complex patterns without deep regex knowledge.

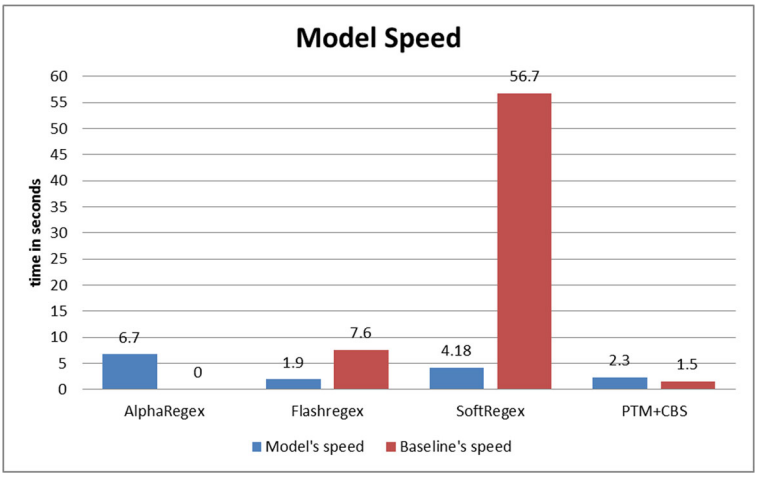

Fig. 8 Speed of various models

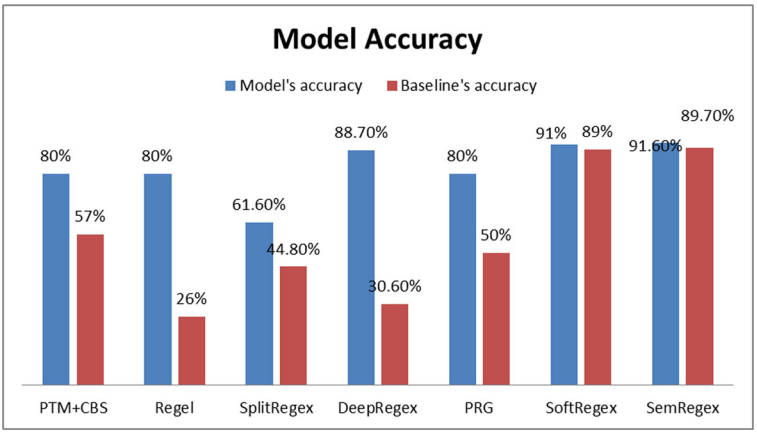

Fig. 9 Accuracy of various models

2. LLMs can also validate and debug regex patterns by identifying common errors and testing them against sample data.
3. In IDEs, they offer real-time suggestions and completions, adapting patterns based on context.
4. Additionally, LLMs can explain regex patterns in plain language and generate comprehensive documentation, enhancing user understanding.
5. They are also capable of dynamically adapting patterns based on evolving data formats and creating domain-specific regexes tailored to particular industries or user needs.

Overall, LLMs can streamline regex development, improve accuracy, and accelerate pattern creation.

4.1 Large language models (LLMs) and generative pre-trained transformers (GPT)

Large language models (LLMs) are advanced AI systems designed to process and generate human language by analyzing vast amounts of text data. These models leverage deep learning techniques to perform a variety of language-based tasks, such as text completion, translation, and code generation. By understanding the intricate details of language—ranging from grammar and syntax to context and nuance—LLMs can produce coherent and contextually appropriate responses.

Generative Pre-trained Transformers (GPT) represent a specific category of LLMs developed by OpenAI. Built on the Transformer architecture, GPT models are pre-trained on extensive text datasets to internalize language patterns and structures. This pre-training equips them with the ability to generate highly coherent and contextually relevant text. Notably, GPT-3, one of the most advanced iterations, excels in producing human-like text and adapting to various contexts, making it an invaluable tool for natural language processing (NLP) applications.

In the realm of program synthesis, LLMs can generate and suggest code snippets, such as regex, based on NL descriptions. This capability helps developers streamline repetitive coding tasks and improve productivity. GPT models further enhance this process by interpreting user intentions expressed in natural language and providing efficient regex solutions for pattern matching and text processing. Let us consider a few current trendy studies which deal with both.

4.1.1 Syncode: ensuring compliance with context-free grammar

LLMs often face challenges when generating outputs that adhere to specific formats governed by context-free grammar (CFG), such as JSON or various programming languages. To tackle this issue, [56] introduce SynCode, a framework designed to ensure that generated outputs comply with CFG rules. By using a deterministic finite automaton (DFA)-based lookup table, SynCode effectively filters valid tokens and significantly reduces syntax errors. This method has demonstrated a reduction in errors by 96.07% for Python and Go code, while also ensuring reliable JSON formatting.

4.1.2 Lilo framework: integrating LLMs with symbolic abstraction

In addressing the limitations of LLMs in complex problem-solving, [17] presents Lilo, a neurosymbolic framework that merges LLMs with symbolic abstraction. Lilo incorporates an LLM synthesizer, a symbolic compression module, and an auto-documentation (AutoDoc) feature. This integration allows Lilo to surpass the performance of DreamCoder in terms of speed while maintaining similar computational costs. Lilo illustrates how LLMs can be leveraged to enhance software problem-solving through a combination of symbolic and neural approaches.

4.1.3 Training engineers in digital design

The training of new engineers in digital design using electronic design automation (EDA) tools like Quartus Prime and Vivado presents challenges, especially when dealing with compile-time synthesis errors. Qui in [44] explore the potential of LLMs to provide beginner-friendly explanations for these errors. Their findings indicate that 71% of the explanations

generated by LLMs are both accurate and helpful, making LLMs a valuable resource for educational purposes in digital design.

4.1.4 Innovations in program synthesis: REI and LTL learners

Valizadeh et al. [59] introduces significant advancements in program synthesis with the development of a fast Regular Expression Inference (REI) algorithm utilizing bitvector matrices, which achieves an impressive speedup of over 1000x on GPUs. Additionally, a new GPU-based Linear Temporal Logic (LTL) learner has been introduced, outperforming existing methods by up to 16,000x. These innovations highlight the potential of GPUs to revolutionize program synthesis by greatly enhancing speed and efficiency.

4.1.5 Enhancing explanatory name accuracy

Nazari et al. [36] propose a method to improve the accuracy of explanatory names generated by LLMs. By validating input-output data and refining prompts, their approach boosts accuracy from 24% to 79%. User studies reveal a strong preference for their method over traditional techniques, underscoring the effectiveness of their approach in enhancing the quality of LLM-generated explanations.

4.1.6 EvoSuiteC3: improving test input readability

In [67], the authors introduce the Context Consistency Criterion (C3) to evaluate the readability of test inputs in relation to source code contexts. Their tool, EvoSuiteC3, generates test inputs with a readability score of 90% for string-type inputs, significantly surpassing the 8% readability achieved by traditional tools. This advancement demonstrates the potential for LLMs to improve the quality and clarity of test inputs.

4.1.7 ACDC (automating code documentation)

Procko and Collins [43] develop the Automatic Code Documentation with Compilers (ACDC) system, which integrates GPT-3.5 with abstract syntax trees (ASTs) in a continuous integration/continuous deployment (CI/CD) pipeline. This system automates real-time documentation generation from Git repositories. Developer feedback highlights a preference for GPT-3.5 due to its concise documentation capabilities, illustrating its effectiveness in automating code documentation processes.

4.1.8 SyntheT2C: enhancing LLM performance with knowledge graphs

Zhong et al. [66] propose SyntheT2C, a method that enhances LLM performance by integrating Knowledge Graphs. This approach addresses the challenge of converting NL to Cypher queries by generating synthetic Query-Cypher pairs. SyntheT2C significantly improves LLM performance in the Text2Cypher task, demonstrating how the integration of Knowledge Graphs can enhance the accuracy and effectiveness of LLMs.

4.2 Top models for program synthesis

The RTI algorithm and LTL learner stand out as leading models in program synthesis. The RTI algorithm achieves an impressive speedup of over 1000 times on GPUs, demonstrating exceptional efficiency in regex inference. The LTL learner, with its remarkable performance improvement of up to 16,000 times, showcases significant advancements in handling complex program synthesis tasks. These models represent the forefront of innovation in program synthesis, offering substantial enhancements in speed and accuracy.

5 Characteristics of regex synthesis and repair frameworks

From the rigorous survey performed in the previous section, we can learn the following characteristics to distinguish between regex syntheses methods,

5.1 Domain agnostic versus domain specific program synthesis

The problem of program synthesis from text can be further subdivided into two main categories: domain agnostic and domain specific. Approaches that generate programs from given textual descriptions irrespective of the output language are called domain agnostic. Nevertheless, when model is able to synthesize program in certain languages only, it is called domain specific program synthesis. For example, model that produces regex patterns may or may not be able to produce CSS selectors script for same NL description.

5.2 Anti-REDOS regex synthesis

Many researches around the problem have an inclination to accompany regex repair mechanism (letting the model produce valid and invalid regexes, converting invalid into valid ones) in addition to synthesis thus producing a complete framework. REDOS proneness [29] is a big issue which leads to repair eventually.

It has been observed that slow and complicated regexes are prone to attack by hackers due to their super liner worst-case complexity. The attack is named as REDOS attack which mean regex-based denial of service attack. It consumes the computing resources of the target machines very effectively if invoked with malign input strings given through user-friendly interface whose goal is to input user intent in a better manner.

Modern tools do not pay attention to generating anti-REDOS regexes rather they are focused on generating the right regex only. Even repair tools focus on correcting incorrect regexes only.

5.3 Model evaluation methods

There are two methods to determine similarity: (1) String equivalence(all their example strings are equal) and (2) minimal DFA equivalence (their DFAs have same semantics) Zhong et al. [64]. If all strings produced by the two regexes in question are same then they are considered string equivalent or syntactically equivalent. This is clearly a bad idea in terms of time complexity. DFAs on the other hand deal with matching minimal DFA of the two regexes in question. This comparison saves time and is judge's equivalence semantically. It

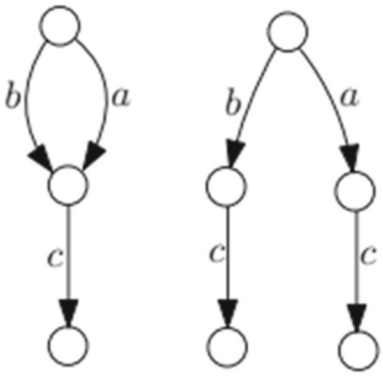

Fig. 10 DFA equivalence for (**a.b + c**)

is preferred to use semantic correctness and therefore, DFA-based equivalence. Figure 10 displays two DFAs for one regex.

6 Challenges faced by regex synthesis model

Synthesis platforms faced many problems, some inherent to their nature, whereas others based on external entities such as dataset and inputs applied.

6.1 Issues in user generated NL text descriptions

In user generated example string, there are two main issues: they are not informative enough and offer massive search space. Most studies aim to infer a probabilistic regular grammar efficiently using novel stochastic process recognition model. The research work [37] prefers incrementally growing a grammar using positive and negative string examples. It uses programming by example method which synthesize rules from input output pairs.

6.2 Shortage of training data

The number and quality of annotated data bind NLP tasks, but there is often a shortage of such training data. In [33] a critical question is asked and then answered **: "Can we combine a neural network (NN) with regular expressions (RE) to improve supervised learning for NLP?".** They developed novel methods to exploit the rich expressiveness of REs at different levels within a NN. Experimental results showed that it is highly effective in exploiting the available small training data, giving a clear boost to the RE-unaware NN.

6.3 Inefficient equivalence determination

With passage of time, more accurate and efficient regex synthesis methods have been invented. The regex synthesis problem has p-space complexity and therefore improvement of its efficiency becomes a major research problem in itself. Main causes of errors are ambiguity of NL (28% failed samples are due to this issue), false prediction for wildcards and quantifiers (35% failed samples because their semantics change according to their position in the regex)

Table 5 Two semantically equivalent but syntactically apart regexes

Text description	Regex	Semantically equivalent regex
three letter word starting with 'X'	\bX[A-Za-z]{2}\b	[A-Za-z]{3})&(\b[A-Za-z] + \b)&(X.*)

and false prediction for keywords (17% samples failed). Almost 20 % failed samples do not lie in any bigger category discussed above Zhong et al. [64]. An example of two semantically equivalent but syntactically apart regexes for same text description is show in Table 5.

6.4 Program aliasing

Another challenge that comes into action here is called program aliasing. It means a semantically equivalent program may have many different syntactically equivalent forms, so accepting one form as ground truth and rejecting all other correct forms is a limitation of syntax-based approaches. **A critical question arises here that how does the model know which regex is the best representative of the given text?** Technically, all candidate regexes are matched to the ground truth regex one by one. This exhaustive search for the best solution is time intensive. Some models even take user feedback to halt the matching process earlier if they are satisfied with the current solution. Sometimes, ranking and post-pruning of candidates is performed to reduce the massive search space.

Finding best regex among all candidates is a computationally intensive challenge. Solution used here is beam search instead of greedy approach. With beam size K, model generates K best candidates and tests whether at least one out of them is the correct one and/or whether the one with highest likelihood is the correct one. Larger value of K will cause more correct regex. Nonetheless, equivalence determination tends to slow down model performance and therefore needs to be handled.

6.5 Issues due to PBE

PBE faces difficulty in learning string classifier that is learning small sub set of regexes from human generated positive and negative strings. Inferring regex methods have to deal with user-supplied string's uninformative nature, thus generating equally believable hypothesis or one that is far away from the ground truth/intended regex. However, it is informative but still computationally inefficient with a huge set of possible regexes.

6.6 Nature of dataset

Two main distinguishing elements of the realistic dataset are: (1) complexity of regex and (2) complexity of NL.

- In regexlib[3] (realistic dataset) average regex length > 40 is common, whereas for other two synthetic datasets average regex length is between 10–40 characters.
- Absence of certain characters within patterns is more likely in synthetic datasets.
- Real-world dataset has been found to possess more complex NL descriptions with word count > 20 whereas for synthetic ones word count < 20.

[3] https://regexlib.com/?AspxAutoDetectCookieSupport=1.

Complexity, density and nature (synthetic or realistic) of dataset play vital role in the model performance. Synthetic datasets are mostly simple, dense, manually paraphrased and annotated, whereas realistic dataset is large scale, sparse and complex. Even various contemporary, high-tech techniques flatly fall-short given some realistic dataset instead of synthetic one.

7 Theoretical framework for downstream task

Applying our review results to hypothetical experimental settings will enable precise evaluation and optimization of regex synthesis methods under controlled conditions. To develop an effective framework for synthesizing regex, a balance between speed and accuracy is crucial. This framework is designed to integrate the strengths of the most advanced models identified in recent research, ensuring optimal performance for various downstream tasks such as data validation and pattern extraction.

Speed Optimization:

FlashRegex is the standout model for rapid regex generation. It reduces the average synthesis time to just 1.9 s, making it highly suitable for scenarios requiring quick regex creation.
SoftRegex also significantly improves training times, achieving up to an 8.8-fold reduction compared to older methods. While it might offer slightly less accuracy, it is a viable option when speed is critical, and a small trade-off in accuracy is acceptable.

Accuracy Optimization:

DeepRegex excels in accuracy, providing a 58% improvement over its baseline. This model is ideal for applications where high precision is essential.
TransRegex achieves up to 38% higher accuracy than other NLP-based methods, making it effective for more complex challenges.
PTM + CBS balance both speed and accuracy, offering a 23% improvement over the baseline Regel while maintaining fast convergence. This model is suitable for tasks requiring both high accuracy and reasonable speed.

Framework Implementation:

To create an effective regex synthesis process, a hybrid approach should be employed:

1. **Rapid Initial Generation:** Start with **FlashRegex** to quickly generate a broad set of potential regex patterns. This phase focuses on speed, allowing for the swift creation of numerous candidate patterns.
2. **Accuracy Refinement:** Apply **DeepRegex** or **TransRegex** to refine and validate the initial patterns. This second stage is dedicated to enhancing accuracy and handling complex patterns that may have been missed initially.
3. **Iterative Improvement:** Establish a feedback loop where regex patterns generated in the initial stage are evaluated against real-world data. Patterns that do not meet accuracy standards should be refined using **DeepRegex** or **PTM + CBS** through iterative adjustments.

Dataset considerations:

Use of a balanced dataset that includes both synthetic and real-world examples has been suggested. This approach **addresses** the limitations of purely synthetic datasets and ensures robustness in diverse scenarios. Datasets such as KB13 can be utilized for high-performance benchmarks, while NL-RX can be employed to evaluate the models against unique regex patterns.

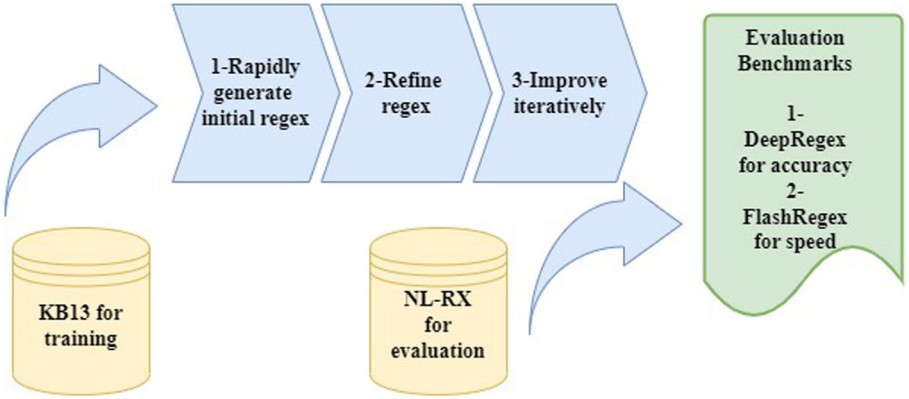

Fig. 11 Theoretical framework for ARS

Evaluation and maintenance:
Measure **performance** based on both speed and accuracy. **FlashRegex** should be used for initial generation, with **DeepRegex** or **TransRegex** used for refinement. Continuous evaluation against benchmark datasets and iterative improvements based on real-world performance are essential for maintaining high performance. Figure 11 graphically presents this hypothetical framework.

In conclusion, this theoretical framework integrates the best practices from both speed-focused and accuracy-focused regex synthesis methods. By combining rapid initial generation with precise refinement and leveraging diverse datasets, the framework aims to achieve optimal performance for regex synthesis tasks, ensuring a balance between speed and precision to meet practical application needs.

8 Conclusions and future recommendations

The significance of regular expressions in various text-processing tasks is undeniable. Yet their automatic **generation** given merely a user written text snippet is hard challenge for even skilled programmers. Moreover, the automatic regex synthesis problem has been poorly studied in terms of literature overtime. We aimed to explore automatic regex synthesis methods, their contributions and limitations. Equipped with a deeper insight into the available methods, the superior goal of correct and efficient regex generation can be achieved. The eminent need of an appropriate dataset with the capability to engage most of these methods for improved outcomes has been elaborated. We also came to know how complexity of queries and regex are related to each other and how models trained for synthetic datasets fail on real datasets.

In future, neural net and semantic similarity-based models might love to tackle more challenging problems in broader families of formal languages, such as mapping between language description and program scripts.

Author contribution Selection of papers,write-up and sectioning by Sadia Tariq Idea, review, proof reading and figures by Dr. Toqir A. Rana.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Bahdanau D, Cho K, Bengio Y (2014) Neural machine translation by jointly learning to align and translate. arXiv:1409.0473.
2. Bartoli A, De Lorenzo A, Medvet E, Tarlao F (2014) Playing regex golf with genetic programming. In: Proceedings of the 2014 annual conference on genetic and evolutionary computation (pp. 1063–1070).
3. Bartoli A, De Lorenzo A, Medvet E, Tarlao F (2016a) Inference of regular expressions for text extraction from examples. IEEE Trans Knowl Data Eng 28(5):1217–1230
4. Bartoli A, De Lorenzo A, Medvet E, Tarlao F (2016b) Regex-based entity extraction with active learning and genetic programming. ACM SIGAPP Appl Comput Rev 16(2):7–15
5. Beal M, Ghahramani Z, Rasmussen C (2001) The infinite hidden Markov model. Adv Neural Info Process Syst, 14
6. Becchi M, Cadambi S (2007) Memory-efficient regular expression search using state merging. In: IEEE INFOCOM 2007-26th IEEE international conference on computer communications, (pp. 1064–1072). IEEE.
7. Bex GJ, Neven F, Schwentick T, Vansummeren S (2010) Inference of concise regular expressions and DTDs. ACM Trans Database Syst (TODS) 35(2):1–47
8. Chen Q, Wang X, Ye X, Durrett G, Dillig I (2020). Multi-modal synthesis of regular expressions. In: Proceedings of the 41st ACM SIGPLAN conference on programming language design and implementation, (pp. 487–502).
9. Chen SF (1995). Bayesian grammar induction for language modeling. arXiv preprint cmp-lg/9504034.
10. Chen Q, Banerjee A, Demiralp Ç, Durrett G, Dillig I (2023) Data extraction via semantic regular expression synthesis. Proc ACM Program Lang 7(OOPSLA2):1848–1877
11. Chida N, Terauchi T (2023) Repairing regular expressions for extraction. Proc ACM Program Lang 7:1633–1656
12. Davis JC, Coghlan CA, Servant F, Lee D (2018) The impact of regular expression denial of service (ReDoS) in practice: an empirical study at the ecosystem scale. In: Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering (pp. 246–256).
13. de Almeida Farzat A, de Oliveira Barros M (2022) Automatic generation of regular expressions for the Regex Golf challenge using a local search algorithm. Genet Program Evolvable Mach 23(1):105–131
14. Devlin J, Zbib R, Huang Z, Lamar T, Schwartz R, Makhoul J (2014). Fast and robust neural network joint models for statistical machine translation. In: Proceedings of the 52nd annual meeting of the association for computational linguistics (Volume 1: Long Papers), (pp. 1370–1380).
15. Fernau H (2009) Algorithms for learning regular expressions from positive data. Inf Comput 207(4):521–541
16. Freydenberger DD, Kötzing T (2015) Fast learning of restricted regular expressions and DTDs. Theory Comput Syst 57(4):1114–1158
17. Grand GJ (2023) Discovering abstractions from language via neurosymbolic program synthesis (Doctoral dissertation, Massachusetts Institute of Technology).
18. Gulwani S, Marron M (2014) Nlyze: interactive programming by natural language for spreadsheet data analysis and manipulation. In: Proceedings of the 2014 ACM SIGMOD international conference on management of data (pp. 803–814).
19. Gvero T, Kuncak V (2015) Synthesizing Java expressions from free-form queries. In: Proceedings of the 2015 ACM SIGPLAN international conference on object-oriented programming, systems, languages, and applications
20. Herzig J, Berant J (2019) Don't paraphrase, detect! Rapid and effective data collection for semantic parsing. arXiv preprint arXiv:1908.09940.
21. Kim SH, Cheon H, Han YS, Ko SK (2021) SplitRegex: faster regex synthesis via neural example splitting.
22. Kim SH, Cheon H, Han YS, Ko SK (2022). Neuro-Symbolic regex synthesis framework via neural example splitting. arXiv preprint arXiv:2205.11258.
23. Kim SH, Im H, Ko SK (2021) Efficient enumeration of regular expressions for faster regular expression synthesis. In: International conference on implementation and application of automata (pp. 65–76). Cham: Springer International Publishing.

24. Kirrage J, Rathnayake A, Thielecke H (2013) Static analysis for regular expression denial-of-service attacks. In: International conference on network and system security (pp. 135–148).
25. Kushman N, Barzilay R (2013) Using semantic unification to generate regular expressions from natural language.
26. Lee M, So S, Oh H (2016) Synthesizing regular expressions from examples for introductory automata assignments. In: Proceedings of the 2016 ACM SIGPLAN international conference on generative programming: concepts and experiences (pp.70–80).
27. Li G, Yang J, Gama J, Natwichai J, Tong Y (2019) Database systems for advanced applications: 24th International Conference, DASFAA 2019, Chiang Mai, Thailand, April 22–25, 2019, Proceedings, Part I (Vol. 11446). Springer.
28. Li Y, Li S, Xu Z, Cao J, Chen Z, Hu Y, Cheung SC (2021) TransRegex: multi-modal regular expression synthesis by generate-and-repair. In: 2021 IEEE/ACM 43rd international conference on software engineering (ICSE)) (pp. 1210–1222). IEEE.
29. Li Y, Xu Z, Cao J, Chen H, Ge T, Cheung SC, Zhao H (2020) FlashRegex: deducing anti-ReDoS regexes from examples. In: 2020 35th IEEE/ACM International conference on automated software engineering (ASE) (pp. 659–671).
30. Lin CH, Liu CH, Chang SC (2011) Accelerating regular expression matching using hierarchical parallel machines on GPU. In: 2011 IEEE global telecommunications conference-GLOBECOM 2011 (pp. 1–5). IEEE.
31. Lin XV, Wang C, Zettlemoyer L, Ernst MD (2018) NL2Bash: a corpus and semantic parser for natural language interface to the Linux operating system. In: Proceedings of the eleventh international conference on language resources and evaluation (LREC 2018).
32. Locascio N, Narasimhan K, DeLeon E, Kushman N, Barzilay R (2016) Neural generation of regular expressions from natural language with minimal domain knowledge. In: Proceedings of the 2016 conference on empirical methods in natural language processing (pp. 1918–1923).
33. Luo B, Feng Y, Wang Z, Huang S, Yan R, Zhao D (2018) Marrying up regular expressions with neural networks: a case study for spoken language understanding. arXiv preprint arXiv:1805.05588.
34. Manshadi M, Gildea D, Allen J (2013) Integrating programming by example and natural language programming. In: Proceedings of the AAAI conference on artificial intelligence 27(1): 661–667.
35. McClurg J, Claver M, Garner J, Vossen J, Schmerge J, Belviranli ME (2022) Optimizing regular expressions via rewrite-guided synthesis. In: Proceedings of the international conference on parallel architectures and compilation techniques (pp. 426–438).
36. Nazari A, Chattopadhyay S, Swayamdipta S, Raghothaman M (2024) Generative explanations for program synthesizers. arXiv preprint arXiv:2403.03429.
37. Ouyang L (2018) Bayesian inference of regular expressions from human-generated example strings. arXiv preprint arXiv:1805.08427.
38. Pan R, Hu Q, Xu G, D'Antoni L (2019) Automatic repair of regular expressions. In: Proceedings of the ACM on programming languages, 3(OOPSLA), 1–29.
39. Park JU, Ko SK, Cognetta M, Han YS (2019) Softregex: generating regex from natural language descriptions using softened regex equivalence. In: Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)) (pp. 6425–6431).
40. Pertseva E, Barbone M, Rudek J, Polikarpova N (2022) Regex+: synthesizing regular expressions from positive examples. In: 11th workshop on synthesis.
41. Petsios T, Zhao J, Keromytis AD, Jana S (2017). Slowfuzz: automated domain-independent detection of algorithmic complexity vulnerabilities. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security (pp. 2155–2168).
42. Pfau D, Bartlett N, Wood F (2010) Probabilistic deterministic infinite automata. Adv Neural Info Process Syst, 23.
43. Procko TT, Collins S (2024) Automatic code documentation with syntax trees and GPT.
44. Qiu S, Tan B, Pearce H (2024). Explaining EDA synthesis errors with LLMs. arXiv preprint arXiv:2404.07235.
45. Rahmani, K., Raza, M., Gulwani, S., Le, V., Morris, D., Radhakrishna, A., Tiwari, A. (2021). Multi-modal program inference: a marriage of pre-trained language models and component-based synthesis. In: Proceedings of the ACM on programming languages, 5(OOPSLA), 1–29.
46. Rathnayake A (2015) Semantics, analysis and security of backtracking regular expression matchers [University of Birmingham].
47. Rathnayake A, Thielecke H (2014) Static analysis for regular expression exponential runtime via substructural logics. CoRR abs/1405.7058.

48. Raza M, Gulwani S, Milic-Frayling N (2015) Compositional program synthesis from natural language and examples. In: Proceedings of the 24th international conference on artificial intelligence (pp. 792–800).
49. Rebele T, Tzompanaki K, Suchanek FM (2018) Adding missing words to regular expressions. In: Advances in knowledge discovery and data mining: 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3–6, 2018, Proceedings, Part II 22 (pp. 67–79). Springer International Publishing.
50. Redd D, Gibson B, Murtaugh MA, Goulet J, Zeng-Treitler Q (2018) Extract clinical measurement values using a regular expression pattern discovery algorithm vs support vector machine. E-Health 2018 ICT, Society Human Beings 2018, 29.
51. Shen Y, Jiang Y, Xu C, Yu P, Ma X, Lu J (2018) ReScue: crafting regular expression DoS attacks. In: 2018 33rd IEEE/ACM international conference on automated software engineering (ASE) (pp. 225–235).
52. Singh R, Gulwani S (2012) Learning semantic string transformations from examples. arXiv preprint arXiv:1204.6079.
53. Sugiyama S, Minamide Y (2014) Checking time linearity of regular expression matching based on backtracking. Info Media Technol 9(3):222–232
54. Sullivan B (2010) New tool: SDL regex fuzzer.
55. Sutskever I, Vinyals O, Le QV (2014) Sequence to sequence learning with neural networks. Adv Neural Info Process Syst, 27.
56. Tariq S, Rana TA (2024) Structure and design of multimodal dataset for automatic regex synthesis methods in Roman Urdu. Int J Data Sci Anal. https://doi.org/10.1007/s41060-024-00612-y
57. Ugare S, Suresh T, Kang H, Misailovic S, Singh G (2024) Improving LLM code generation with grammar augmentation. arXiv preprint arXiv:2403.01632.
58. Uma M, Sneha V, Sneha G, Bhuvana J, Bharathi B (2019). Formation of SQL from natural language query using NLP. In: 2019 international conference on computational intelligence in data science (ICCIDS) (pp. 1–5). IEEE.
59. Vaithilingam P, Pu Y, Glassman EL (2023) The usability of pragmatic communication in regular expression synthesis. arXiv preprint arXiv:2308.06656.
60. Valizadeh M (2024) Program synthesis on GPUs University of Sussex.
61. Wang Y, Berant J, Liang P (2015) Building a semantic parser overnight. In: Proceedings of the 53rd annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing (Volume 1: Long Papers) (pp. 1332–1342).
62. Wüstholz V, Olivo O, Heule MJ, Dillig I (2017) Static detection of dos vulnerabilities in programs that use regular expressions (extended version). arXiv preprint arXiv:1701.04045.
63. Yu X, Becchi M (2013) GPU acceleration of regular expression matching for large datasets: exploring the implementation space. In: Proceedings of the ACM international conference on computing frontiers (pp. 1–10).
64. Zhang T, Lowmanstone L, Wang X, Glassman EL (2020) Interactive program synthesis by augmented examples. In: Proceedings of the 33rd annual ACM symposium on user interface software and technology (pp. 627–648).
65. Zhong Z, Guo J, Yang W, Peng J, Xie T, Lou JG, Zhang D (2018a) SemRegex: a semantics-based approach for generating regular expressions from natural language specifications. In: Proceedings of the 2018 conference on empirical methods in natural language processing.
66. Zhong Z, Guo J, Yang W, Xie T, Lou JG, Liu T, Zhang D (2018b) Generating regular expressions from natural language specifications: are we there yet? In: Workshops at the thirty-second AAAI conference on artificial intelligence.
67. Zhong Z, Zhong L, Sun Z, Jin Q, Qin Z, Zhang X (2024) SyntheT2C: generating synthetic data for fine-tuning large language models on the Text2Cypher task. arXiv preprint arXiv:2406.10710.
68. Zhou Z, Tang Y, Lin Y, He J (2024) An LLM-based readability measurement for unit tests' context-aware inputs. arXiv preprint arXiv:2407.21369.

Sadia Tariq serves as a Lecturer at the University of Engineering and Technology (UET), Lahore (Narowal campus) in Punjab, Pakistan. She completed her B.Sc. computer engineering degree from UET Lahore, Pakistan, in 2007. She completed her M.Sc. computer engineering degree from UET Lahore, Pakistan, in 2011. She has been pursuing her PhD degree from The University of Lahore, Pakistan, since 2020. Sadia's research interests include data mining, data modeling, natural language processing, and machine learning.

Toqir A. Rana completed his PhD from Universiti Sains Malaysia. He has more than 12 years of academic experience in several universities. He is currently working as Associate Professor at Computer Science and IT Department, The University of Lahore, Pakistan. His research interests are data mining, text mining, sentiment analysis, and NLP. He has published several articles in top journals and international conferences. He is also serving as academic editor in Plos One journal and reviewer of several top ranked international journals. He has supervised several Ph.D. and master students for their research thesis.